# enable_prompt_caching.md — § . . Measure M (prompt-cache the governance prefix)

| Field | Value |
| --- | --- |
| Revision | 1 |
| Last modified | 2026-06-09T00:00:00Z |
| Status | active |
| Authority | constitution submodule, universal (§11.4.17). Measure **M1** of the §11.4.141 token-efficiency mandate — the PRIMARY, safest, biggest lever. |

> **What this is.** The exact, **reversible**, **per-agent** procedure to prompt-cache the *static governance prefix* (consumer `CLAUDE.md` / `AGENTS.md` / `QWEN.md` + `constitution/*` ). M1 is **transparent**: the model receives byte-identical input cached or not, so it **removes no rule, weakens no gate, changes no verdict** — it changes only **billing** (cache reads cost ~0.1× base input price). The only failure mode is a *silent invalidator* (a per-turn timestamp / UUID / unsorted-JSON rendered **ahead** of the breakpoint) which costs money but never corrupts and is detectable via `cache_read_input_tokens` . Verify with `scripts/token_accounting.sh` and the companion helper `scripts/enable_prompt_caching_check.sh` .

---

## . The one invariant M    depends on

**A byte-stable governance prefix with the cache breakpoint at its end, and nothing volatile rendered ahead of it.** Everything below is just how each agent expresses that invariant. The *project's* job (whatever the agent) is the **don't-invalidate discipline**:

- governance files stay byte-identical within a session;
- no per-turn timestamp / UUID / unsorted-JSON / changing tool-list is injected ahead of the breakpoint.

A silent invalidator collapses `cache_read_input_tokens` toward 0 — that is the mechanically-detectable signature the §1.1 paired mutation exploits (MEASUREMENT §8).

---

## . Claude Code — native (no API code needed)

Claude Code **caches the system + instruction prefix natively**; you do not write `cache_control` yourself. The project responsibility is to keep the prefix stable and to *confirm the cache is warm and free of pre-breakpoint volatiles*.

**Enable / confirm (reversible — nothing is changed; you only observe):**

1. Run any normal turn, capturing the result JSON:

   ```
   claude -p "say ok" --output-format json > /tmp/cc_turn.json
   ```

2. Confirm warmth on the **second** governance-bearing turn (the first writes the cache, the second reads it):

   ```
   bash scripts/enable_prompt_caching_check.sh --transcript /tmp/cc_turn.json
   ```

   PASS iff `cache_read_input_tokens > 0` on a steady-state turn.

3. **1-hour TTL posture for gap-heavy sessions** (optional; set by the harness/host, not by editing governance): use the host's documented 1h-cache setting so a >5-minute gap between turns does not force a full re-write. The project keeps the prefix stable; the TTL is a host knob.

**Reverse / disable (for an A/B BEFORE run):** run the workload with the governance **inlined every turn and caching effectively cold** — e.g. on a fresh session per turn, or by deliberately injecting a per-turn volatile ahead of the prefix (this is exactly the §1.1 mutation, see MEASUREMENT §8). No governance file is edited; the BEFORE state is a *posture*, fully reversible.

**Do NOT** add a per-turn timestamp/UUID, an unsorted JSON blob, or a turn-varying tool list ahead of the governance. That is the silent invalidator.
[2]

---

## . Anthropic-SDK driver (Cursor / Aider / any custom Messages-API caller)

Put the governance in `system` as a single (or final) text block carrying `cache_control: {"type":"ephemeral"}`, and **reuse the exact same prefix bytes on every request**.

```
// Messages API request (per the claude-api skill shared/prompt-caching.md):
{
  "model": "claude-opus-4-8",
  "system": [
    {
      "type": "text",
      "text": "<the FULL static governance: consumer CLAUDE.md + constitution/*>",
      "cache_control": { "type": "ephemeral" }   // 5-min TTL; use {"type":"ephemeral","ttl":"1h"} for gap-heavy sessions
    }
  ],
  "messages": [ /* the per-turn, volatile content goes HERE, AFTER the breakpoint */ ]
}
```

**Verify:** make the same request twice; on the second, assert `usage.cache_read_input_tokens > 0` :

```
# write each response's usage object to a JSONL transcript, then:
bash scripts/enable_prompt_caching_check.sh --transcript driver_usage.jsonl
```

**Reverse:** drop the `cache_control` field (or change the prefix bytes each turn). Fully reversible, zero behaviour change either way — only the bill differs.

**Pitfalls (all are silent-invalidators):** a non-deterministic system block (timestamp, request id, unsorted map), reordered tool definitions, or any volatile content placed *before* the cache breakpoint.

3

---

## . Gemini CLI / Qwen Code

Use **API-key auth** so the provider's built-in context cache engages, and keep the system instruction **stable** across calls.

- **Gemini:** API-key mode + a stable `systemInstruction` ; confirm via the response usage/cache-hit telemetry ( `cachedContentTokenCount` > 0 on a repeated context).
- **Qwen:** the equivalent context-cache via API-key auth + stable system prompt.

**Honest N/A:** if the agent is run in a mode that exposes no cache telemetry, M1 is **SKIP-with-reason** for that agent per §11.4.3 (never a fake PASS) — record the reason; do not claim a warm cache you cannot observe.

---

## . Verification + reversal summary

| Agent | Enable | Confirm warm | Reverse |
|---|---|---|---|
| Claude Code | native (keep prefix stable) | `cache_read_input_tokens>0` on 2nd governance turn | run cold / inject pre-breakpoint volatile (=§1.1 mutation) |
| SDK driver | `cache_control:{type:ephemeral}` on the system governance block | 2nd identical-prefix request `cache_read>0` | drop `cache_control` / vary the prefix |
| Gemini / Qwen | API-key auth + stable systemInstruction | `cachedContentTokenCount>0` (or honest SKIP) | non-API-key mode / vary system prompt |

Every row is **transparent** — no rule, gate, or verdict changes; only the per-token price of the (byte-identical) governance prefix changes.

5 11 4 28

---

## . Decoupling / reuse (§ . . )

M1 is a **property of prompt assembly** (stable prefix + breakpoint), not project code. Any consuming project inherits it by keeping *its own* governance prefix stable and breakpointed, and supplying *its own* model ids to the harness. Zero ATMOSphere / project-specific value is required here.

---

## Cross-references

- §11.4.141 (token-efficiency mandate) · §11.4.6 (measured-not-asserted) · §11.4.69 (captured-evidence) · §11.4.50 (deterministic) · §11.4.3 (SKIP-with-reason) · §1.1 (the silent-invalidator paired mutation).

- `scripts/token_accounting.sh` (the measurement harness) · `scripts/enable_prompt_caching_check.sh` (the warm-cache confirmation helper) · `docs/research/token_efficiency/MEASUREMENT.md` (full proof methodology).
- claude-api skill `shared/prompt-caching.md` (usage-field semantics, 0.1×/1.25×/2× multipliers, silent-invalidator detection).